SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Roshan R	**Reg. No.:**	192524085
Section / Batch:	AI&DS	**Date:**	21/10/2026

Code of Conduct

I Roshan R, 192524085 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Roshan R*

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.
(b) Explain how Hill Climbing would work in this scenario and identify its limitations.
(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.
(b) Analyze the computational challenges of large game trees.
(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
(d) Design an evaluation function and justify the factors considered.
(e) Recommend a strategy for balancing optimality and real-time performance.

ANSWERS TO THE SCENARIO BASED ASSIGNMENT

CSA17 – Artificial Intelligence | CO2 | Assessment Tool 2
Student: Roshan R | Reg. No.: 192524085

Answer 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions)*

Scenario set-up and a small worked example

The drone has to go from the base S to a relief camp G. The map is reduced to six waypoints. The numbers on the edges are the travel costs in minutes after adjusting for wind and terrain, and h(n) is the straight-line estimate to G. The key issues are that roads can get blocked at any moment, edge costs change with the weather, and the heuristic only knows about distance and nothing about flooding.

Edge	S–A	S–B	A–D	D–G	B–C	C–G
Cost g	2	4	3	9	3	4

Node	S	A	B	C	D	G
h(n)	8	4	6	4	3	0
True cost to G	11	12	7	4	9	0

Since h(n) is never larger than the true cost to G in any row, this heuristic is admissible. The two routes are S-A-D-G with cost 2+3+9 = 14 and S-B-C-G with cost 4+3+4 = 11, so the best route costs 11.

(a) Greedy Best-First Search

Greedy search keeps an open list ordered only by h(n), so it always expands the node that looks closest to the destination and ignores the cost already spent. Trace:

Step	Expanded	Open list after expansion (node : h)	Comment
1	S	A:4, B:6	A looks closer to G
2	A	D:3, B:6	D has the smallest h
3	D	G:0, B:6	G is picked next
4	G	–	Goal. Path S-A-D-G, cost = 14

Greedy reached the goal after expanding only 3 nodes, which makes it very fast, but the answer costs 14 minutes against the best 11. The strength is speed and a small memory footprint, which suits a drone with a weak onboard processor. The weaknesses are that it is not optimal and not complete on graphs with loops unless a closed list is kept. Under uncertainty it is also risky. A straight-line heuristic happily leads the drone towards a short but flooded valley, and if D-G gets blocked the drone is stuck at a dead end and has to climb back to B with the time already wasted.

(b) A* search

A* orders the open list by f(n) = g(n) + h(n), where g(n) is the real cost travelled so far and h(n) is the estimate of what is left. This way it prefers a node which is cheap to reach *and* close to the goal, so cost and heuristic are balanced automatically.

Step	Expanded	g	f = g + h	Open list after expansion (node : f)
1	S	0	0 + 8 = 8	A:6, B:10
2	A	2	2 + 4 = 6	D:8, B:10
3	D	5	5 + 3 = 8	B:10, G:14 (g = 14)
4	B	4	4 + 6 = 10	C:11, G:14
5	C	7	7 + 4 = 11	G:11 (g = 11 replaces 14)
6	G	11	11 + 0 = 11	Goal. Path S-B-C-G, cost = 11

A* first went down A and D, as greedy did, but when G appeared with f = 14 the node B had a lower f of 10, so it was expanded and the cheaper route through C was found. The price is 5 expansions instead of 3, and a larger open list. Because the heuristic never overestimates, the first time G is removed from the open list its cost is optimal.

(c) Impact of heuristic accuracy

- **Greedy** depends only on h, so its behaviour is as good as the heuristic. If the straight-line distance ignores flooded areas (h is too optimistic near a blocked region) the drone is pulled into the region again and again, and there is no cost term to correct it.
- **A*** is more forgiving because g keeps growing along a bad path and the other branches become attractive again. Optimality holds only when h is admissible (never overestimates); if it is also consistent, no node has to be re-opened.
- If h = 0 for all nodes, A* turns into uniform-cost search (optimal but slow). If h equals the true cost, A* expands only the nodes on the best path. A better heuristic therefore means fewer expansions. If h overestimates (for example inflating the distance in storm areas without care), A* can miss the best route, so any weather penalty should be added to the edge costs g, not to h.
- Straight-line distance is optimistic whenever the drone must fly around flooded zones, which makes it admissible but not very tight in this scenario.

(d) Effect of dynamic changes (blocked paths)

When an edge is blocked after planning, the computed route is invalid. With greedy search, the drone has usually committed to one branch and has few alternatives stored, so it backtracks and may restart. In the example, if D-G is flooded when the drone reaches D, greedy has used 2 hops (cost 5) and must return to B, giving a total of 5 + 3 + 4 + 3 + 4 = 19 minutes. A* on the same map would have gone to B before reaching G (its open list still held B with f = 10) and would lose much less. Costs that change over time mean g values become stale, so the plan must be repaired rather than computed once. Repeating A* from the current position after each update is simple but wasteful. Incremental methods such as D* Lite or LPA* reuse the previous search and only fix the affected nodes, which fits partial real-time updates.

(e) Recommendation

Recommended algorithm: A* (with replanning when the map changes). Reasons: (1) medicine delivery is life critical, so a good-quality route matters more than the few extra milliseconds greedy saves; (2) A* with an admissible heuristic guarantees the minimum-delay route for the current map, while greedy gave 14 against 11; (3) risk can be put into the edge cost, for example cost = distance × weather factor + penalty for areas with flood risk, so safety is handled directly in g; (4) the graph of waypoints is small enough for real-time use, and if time is tight, weighted A* (f = g + w·h with w slightly above 1) gives a faster answer with a known bound on sub-optimality. Greedy can still be used as a quick fallback path when the drone's battery or time is almost finished.

Answer 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms)

(a) Formulation as an optimization problem

- **State:** a complete timing plan, i.e. for every intersection i the cycle length C_i, the green split for each phase and the offset O_i with respect to its neighbours.
- **Objective:** minimise $F = w_1 \cdot$(total waiting time) + $w_2 \cdot$(fuel consumed) + $w_3 \cdot$(congestion index). The three terms are normalised, and the weights reflect the city's priorities.
- **Constraints:** minimum green time (for example 10 s) for pedestrians, maximum cycle length (for example 120 s), and no conflicting movements shown green together.
- **Neighbour move:** change one value, for example add or remove 5 s of green at one intersection or shift one offset by 5 s.

Traditional search is inefficient for two reasons. First, the space is enormous: if each intersection has 10 cycle lengths, 5 splits and 12 offsets, there are 600 options per junction, and for 300 junctions this gives $600^{300} \approx 10^{833}$ plans, which cannot be enumerated by BFS, A* or any systematic search. Second, the path to a solution does not matter, only the final plan does, and traffic changes every few minutes, so any answer found slowly is already out of date. Local search keeps only the current plan in memory and improves it step by step.

(b) Hill climbing

Hill climbing starts from the current timing plan, evaluates all neighbouring plans and moves to the best one that lowers F. It stops when no neighbour is better. It needs very little memory and responds quickly, which suits continuous adjustment. To see how it can fail, take the green time of one junction and its cost F (lower is better):

Green time (s)	**20**	**25**	**30**	**35**	**40**	**45**	**50**	**55**
Cost F	95	80	70	75	60	50	58	65

Starting at 30 s (F = 70), the neighbours are 25 s (80) and 35 s (75). Both are worse, so hill climbing stops at 30 s even though 45 s (F = 50) is the global best. Limitations: it gets trapped in local optima, it can wander on flat areas, it depends on the starting plan, and it never accepts a temporary worsening.

(c) Local maxima, plateaus and ridges in the traffic setting

- **Local optimum:** the 30 s plan above. Every small change makes things worse, for instance because a longer green here causes a queue at the next junction, yet a much better plan exists elsewhere.
- **Plateau:** at night with very few vehicles, changing a green time by 5 s does not change waiting time or fuel at all, so F is flat and the algorithm has no direction to move.
- **Ridge:** a green wave needs several junctions to shift their offsets together. Changing one offset alone breaks the wave and raises F, even though changing all of them together would lower it. Single-variable moves cannot climb such a ridge.

(d) Simulated annealing and genetic algorithms

Simulated annealing (SA) allows a worse neighbour to be accepted with probability $e^{-\Delta E/T}$, where T is a temperature that is slowly reduced. Using the table above with starting T = 20 from 30 s:

Move	ΔE (new − old)	Acceptance	Result
30 → 35 s	75 − 70 = +5	$e^{-5/20} = 0.78$	Accepted (uphill move allowed)
35 → 40 s	60 − 75 = −15	Better, always accepted	Accepted
40 → 45 s	50 − 60 = −10	Better, always accepted	Accepted, F = 50 (global best)

SA therefore crossed the hill at 35 s that stopped hill climbing. As T falls, uphill moves become rare and it settles in a good region. **Genetic algorithms (GA)** keep a population of complete plans. Each chromosome is the list of green times and offsets of all junctions; fitness = 1/F. Selection chooses good plans, crossover swaps whole districts between two parent plans (this preserves good green waves inside a district) and mutation changes a few genes. Because many plans are explored in parallel, GA is far less likely to be trapped, and it can handle ridges since crossover changes several variables at once.

(e) Recommendation

Recommended: a hybrid approach. Split the city into zones of 10 to 20 junctions. Use a GA (or SA) to find a strong global plan periodically, for example every 15 minutes or when a big change such as an accident occurs, and use hill climbing or SA at low temperature inside each zone for continuous fine tuning every cycle. This is scalable because each zone is optimised independently and in parallel, and the cost of evaluating a plan grows linearly with the number of junctions. It is adaptable because the current plan is used as the starting point (warm start) when traffic changes, so the system does not restart from zero. Plain hill climbing alone is too easily trapped, and GA alone is too slow to react every minute, so the combination gives the best balance.

Answer 3: Autonomous Rover Exploring Mars

Topic: Online search agent

(a) Why an online search agent is needed

An offline planner computes the whole route before moving, so it needs a complete model of the environment. The rover does not have one: the terrain is unknown, craters and rocks are discovered only when the sensors see them, and communication with Earth has a delay of roughly 4 to 24 minutes, so remote control is not possible. The rover must therefore interleave *planning and acting*: choose an action using what is known, execute it, observe, update the model and plan again. This is exactly an online search agent. Also, the exploration itself is the task, as the rover learns the map only by travelling in it.

(b) Challenges of partial observability and dynamic environments

- **Limited sensing:** the rover sees only a few metres around it, so it cannot tell whether the cell ahead leads to a dead end.
- **Noisy percepts:** shadows or dust can make a safe slope look like a rock, or hide a crater edge.
- **Irreversible actions:** falling into a crater or getting stuck in sand cannot be undone. The environment should be safely explorable, meaning the goal remains reachable from every state the rover visits, and the rover must stay within that.
- **Changing environment:** dust storms change visibility and solar power, and loose sand may shift, so earlier observations can become outdated.
- **Resource limits:** battery, memory and wheel wear limit how much wandering is affordable, and a poor competitive ratio (actual path length divided by the best path had the map been known) means wasted energy.

(c) Building and updating the internal model

The rover keeps an occupancy-grid map where each cell is *unknown*, *free* or *hazard* (with a confidence value), plus a table result(s, a) = s′ that records where each action led, and a heuristic table H(s) for the cost to reach the target. After every move it: (1) reads the sensors and marks the visible cells; (2) stores the outcome of the action it just took; (3) revises H(s) of the state it left; (4) chooses the next action from the updated model. Newly discovered hazards are marked permanently, while uncertain cells keep a probability that is updated with each new reading.

To show the update, use the LRTA* rule: in state s choose the action minimising c(s,a) + H(s′), then set H(s) to that minimum. Map: S is connected to X (a dead end the rover cannot see) and to Y; Y connects to Z; Z connects to the sample site G. Every move costs 1. Initial H: S = 2, X = 1, Y = 2, Z = 1, G = 0.

Step	Rover at	Options c + H(s′)	Move to	Updated H(s)
1	S	X: 1+1 = 2, Y: 1+2 = 3	X	H(S) = 2
2	X	only S: 1+2 = 3 (dead end found)	S	H(X) = 3
3	S	X: 1+3 = 4, Y: 1+2 = 3	Y	H(S) = 3
4	Y	S: 1+3 = 4, Z: 1+1 = 2	Z	H(Y) = 2
5	Z	Y: 1+2 = 3, G: 1+0 = 1	G	H(Z) = 1

The rover reached G after 5 moves (S-X-S-Y-Z-G), whereas the best path S-Y-Z-G has 3, so the competitive ratio is 5/3 ≈ 1.67. The wasted trip into X was the price of not knowing the map, and the update H(X) = 3 ensures it never makes that mistake again.

(d) Comparison with uninformed and informed search

Aspect	Uninformed (BFS, DFS, UCS)	Informed offline (Greedy, A*)	Online search (online DFS, LRTA*)
Map needed	Complete map before starting	Complete map and heuristic before starting	Not needed; learned while moving
Where it expands	Any node in the frontier	Any node in the open list	Only the node where the agent physically is
Acting	Plans fully, then acts	Plans fully, then acts	Plans and acts alternately
Dead ends	Found in the model, no cost	Found in the model, no cost	Found by actually going there (real cost)
Optimality	UCS optimal on the known map	A* optimal on the known map	Not guaranteed; measured by competitive ratio
Fit for the rover	Poor	Poor (needs the map)	Good

Offline algorithms can jump from one frontier node to another because it costs nothing in a simulation. A rover cannot teleport, so an online method must expand the neighbouring node and backtrack physically. Online DFS therefore needs reversible actions, and LRTA* adds learning of H values so repeated mistakes are avoided.

(e) Most suitable approach

Recommended: an online informed search such as LRTA* (or online A*/D* Lite on the growing map), with a safety layer. It copes with uncertainty because it needs no prior map and updates H after every step; it adapts because new obstacles change the model immediately and the next decision uses it; and the heuristic (distance to the sample site) gives direction so the rover does not wander as blind online DFS would. For safety, unknown cells are treated as risky and given a high cost, the rover prefers reversible moves and keeps a retreat path to a known safe cell, slopes or rocks above a threshold are never entered, and energy reserve is monitored so that it can return when the battery is low. Learning by trial is acceptable for a cell of cost 1 in the example, but a crater cannot be tried, which is why the safety layer is essential.

Answer 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking)

(a) CSP formulation

- **Variables:** one variable per exam, E_1, …, E_n, i.e. the timetable entry of each course.
- **Domains:** the set of (day, time slot, hall) combinations that exist in the exam period. Each domain is pruned first by unary constraints, such as the faculty being unavailable or a hall being too small.
- **Constraints:** (1) *No overlap:* if two courses share at least one student then slot(E_i) ≠ slot(E_j) (binary). (2) *Hall capacity:* at most H exams (or seats) in one slot, an all-different/global constraint on halls. (3) *Precedence:* day/slot(E_i) < day/slot(E_j) when subject i must come before j. (4) *Faculty:* an examiner or invigilator cannot supervise two halls in a slot and may be unavailable on given days (unary). (5) *Special accommodations:* students needing extra time or a separate room restrict the exam to slots with a free room and a longer gap.

Small example used below. Five exams A (Data Structures), B (Maths), C (Physics), D (AI), E (DBMS) and three slots T1, T2, T3 with 2 halls per slot. Student conflicts: A–B, A–D, B–C, B–D, C–E. Precedence: B before D. Faculty: A cannot be in T3. Accommodation: E only in T1 or T2.

Exam	A	B	C	D	E
Initial domain	{T1, T2}	{T1, T2, T3}	{T1, T2, T3}	{T1, T2, T3}	{T1, T2}

(b) Backtracking search

Backtracking assigns one variable at a time, checks the constraints with already assigned variables and, if a value violates one, tries the next value. If a variable has no legal value left, it returns to the previous variable and changes that choice. Using a poor variable order D, B, A, C, E with values tried in order T1, T2, T3:

Step	Assignment tried	Result
1	D = T1	Allowed so far (nothing else assigned)
2	B = T1, T2, T3	B must be before D = T1: all rejected, so backtrack
3	D = T2, then B = T1	Both allowed
4	A = T1, T2, T3	T1 clashes with B, T2 clashes with D, T3 breaks faculty: dead end, backtrack
5	B has no other value before T2	Backtrack to D
6	D = T3, B = T1, A = T2	All allowed (A–B and A–D differ)
7	C = T1 , C = T2 , E = T1	C clashes with B in T1; E in T1 does not clash with C (T2). Hall check: T1 has B, E = 2 halls, fine

Final timetable: T1 = {B, E}, T2 = {A, C}, T3 = {D}. It is valid but the search wasted work on steps 1 to 5 because the failures were found late. The time grows exponentially with the number of exams in the worst case (timetabling contains graph colouring, which is NP-complete).

(c) Constraint propagation and forward checking

Forward checking removes, after every assignment, the values from the neighbours' domains that conflict with it. If any domain becomes empty, backtrack immediately. With the minimum remaining values (MRV) rule, choosing the variable with the smallest domain first:

Step	Assign	A	B	C	D	E	Remarks
0	start	T1,T2	T1,T2,T3	T1,T2,T3	T1,T2,T3	T1,T2	A and E tie at 2; A has more conflicts, so A first
1	A = T1	T1	T2,T3	T1,T2,T3	T2,T3	T1,T2	T1 removed from B and D (conflict with A)
2	B = T2	T1	T2	T1,T3	T3	T1,T2	T2 removed from C; D must be after B so D = T3
3	D = T3	T1	T2	T1,T3	T3	T1,T2	No unassigned neighbour of D affected
4	C = T1	T1	T2	T1	T3	T2	T1 removed from E (C–E); T1 now has A and C = 2 halls
5	E = T2	T1	T2	T1	T3	T2	T2 has B and E = 2 halls. Solution found, no backtracking

The result is T1 = {A, C}, T2 = {B, E}, T3 = {D}. All constraints are satisfied: no two exams with common students share a slot, B (T2) is before D (T3), A is not in T3, E is in T2 and no slot holds more than 2 exams. In this run forward checking found the solution with zero backtracks, compared with the failure at step 4 of plain backtracking. It also catches dead ends early: in the earlier order, assigning B = T1 would have emptied A's domain at once. Stronger propagation such as arc consistency (AC-3) would additionally have deduced B = T2, D = T3 before any assignment because of the precedence constraint.

(d) Real-world challenges and best strategy for scalability

- **Size:** thousands of courses and tens of thousands of students make the conflict graph large and dense.
- **Over-constrained cases:** sometimes no timetable satisfies everything. Constraints should be split into hard ones (no overlap, hall capacity, precedence) and soft ones (no student with more than two exams a day, gaps between exams), and the soft ones are minimised using a penalty cost.
- **Changes:** late registrations, a hall becoming unavailable or a faculty member falling ill require repair, not full recomputation.
- **Fairness and accommodation:** special-needs students need extra time, rooms near the ground floor or separate invigilation.

Best strategy: backtracking search enhanced with MRV (variable choice), the degree heuristic as a tie-breaker, least-constraining-value ordering, forward checking or AC-3, and conflict-directed backjumping. For very large instances, split the problem by department or by day and solve the parts separately, then use min-conflicts local search to repair the few violations left and to improve the soft constraints. When a change arrives, only the affected exams are unassigned and re-solved, which keeps the rest of the timetable stable.

Answer 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions)

(a) Minimax model of the game

The game is modelled as a two-player zero-sum game. MAX is the AI and MIN is the human opponent. The components are: initial state (the current board with all units, resources and buildings), PLAYER(s) (whose turn), ACTIONS(s) (move, build, attack, research), RESULT(s,a) (the next state), TERMINAL-TEST(s) (a base destroyed or surrender) and UTILITY(s) (+1 win, −1 loss, 0 draw). Minimax assumes the opponent always plays the move that is worst for the AI. At a MAX node it takes the maximum of the children's values, and at a MIN node the minimum, so the AI picks the move whose worst-case result is best. Because a real-time game is continuous and the players act together, it has to be approximated by discretising time into short turns and grouping low-level orders into macro actions.

(b) Computational challenges of large game trees

The number of nodes is about b^d, where b is the branching factor and d the depth in plies. For b = 30 and d = 6 there are $30^6 \approx 7.3 \times 10^8$ leaf states, and for b = 50 and d = 8 there are about 3.9×10^{13}. Evaluating each state also costs time, as armies and economy must be summarised. With a limit of a few tens of milliseconds per decision, full minimax to the end of the game is impossible, memory usage is high, and the same state is reached by many move orders, which wastes effort. So the search must be cut off at a fixed depth and an evaluation function must estimate the value of unfinished positions.

(c) Alpha-beta pruning

Alpha-beta keeps two bounds during the search: α, the best value MAX can already guarantee on the path to the root, and β, the best (lowest) value MIN can guarantee. When $\alpha \geq \beta$ at a node, the remaining children cannot change the final decision, so they are pruned. The result is identical to minimax. Worked example: the root is MAX with three MIN children whose leaves are (3, 12, 8), (2, 4, 6) and (14, 5, 2).

Node visited	α	β	Value / action
Root (MAX)	−∞	+∞	start
MIN 1, leaf 3	−∞	3	β = 3
MIN 1, leaf 12	−∞	3	no change (min stays 3)
MIN 1, leaf 8	−∞	3	MIN 1 returns 3, root α = 3
MIN 2, leaf 2	3	2	β = 2 ≤ α = 3, so **prune leaves 4 and 6**, MIN 2 returns 2
MIN 3, leaf 14	3	14	β = 14
MIN 3, leaf 5	3	5	β = 5
MIN 3, leaf 2	3	2	β = 2, MIN 3 returns 2
Root	3	+∞	Root value = max(3, 2, 2) = **3**, best move is the first child

Only 7 of the 9 leaves were evaluated, and the same answer 3 was obtained as in plain minimax. The reason for pruning leaves 4 and 6 is that MAX already has a move worth 3, and in the second branch MIN can force at most 2, so MAX would never choose that branch whatever the other leaves are. In the best case, when the best moves are examined first, the number of nodes falls from b^d to about $b^{d/2}$. For b = 30, d = 6 this is 30^3 = 27,000 instead of 729,000,000, or alternatively the AI can look twice as deep in the same time. In the worst case (bad move ordering) there is no saving, so ordering moves well (captures and attacks first, previously best moves first, killer moves) is important.

(d) Evaluation function

At the cutoff depth the AI estimates the state with a weighted linear function $\text{Eval}(s) = w_1 f_1 + w_2 f_2 + \dots$ where each feature f is the difference (AI minus opponent) normalised to the range −1 to +1:

Feature	Weight	Why it is included
f_1 Army strength (units weighted by cost, attack and health)	0.35	Fights decide the game, and the army is the most direct measure of power
f_2 Resource income and stockpile	0.20	The economy decides how fast an army can be rebuilt
f_3 Map control (territory, expansions held)	0.15	Gives vision, more resources and safer positions
f_4 Base and key-building health	0.15	Losing the main base ends the game, so it must be weighted
f_5 Technology and production capacity	0.10	Better tech converts into long-term advantage
f_6 Unit positioning / threat to opponent	0.05	Rewards flanks and defended choke points, but is less reliable

Example. Suppose $f_1 = +0.4$, $f_2 = +0.2$, $f_3 = -0.1$, $f_4 = 0.0$, $f_5 = +0.3$, $f_6 = 0.2$. Then $\text{Eval} = 0.35(0.4) + 0.20(0.2) + 0.15(-0.1) + 0.15(0) + 0.10(0.3) + 0.05(0.2) = 0.14 + 0.04 - 0.015 + 0 + 0.03 + 0.01 =$ **0.205**, a small advantage for the AI. A good evaluation function must be fast to compute (it is called at millions of leaves), must agree with the true utility at terminal states (+1 or −1), and should be tuned by playing test games or from recorded matches. A linear combination is chosen because it is cheap and its weights are easy to explain and adjust.

(e) Strategy for balancing optimality and real-time performance

- **Iterative deepening with a time limit:** search to depth 1, then 2, then 3 and so on until the time budget is used, and play the best move of the last completed depth. An answer is always ready, and the best move of the previous depth improves move ordering for the next.
- **Alpha-beta with good move ordering** and a **transposition table** that stores values of states already seen, so repeated positions are not recomputed.
- **Quiescence search** at the cutoff: continue only through unstable moves (ongoing battles) so that the evaluation is not applied in the middle of a fight.
- **Abstraction and selective search (beam):** consider only the best few macro actions per unit group, which lowers b from about 30 to 5-8 in practice.
- **Run the search in a separate thread** from rendering, with a fallback scripted move if time runs out, and give the AI different difficulty levels by limiting the depth or time.

Conclusion: use depth-limited, time-bounded alpha-beta (iterative deepening) with the weighted evaluation function above. It keeps the decisions close to minimax-optimal when time allows and degrades gracefully when time is short.